

CloudOps Capstone

Projektübersicht · Modul-4-Abschlussprojekt · Syntax Institut · Danial Ahmed · Stand: 16. Juli 2026

Eine kleine Aufgaben-/Ticket-Verwaltung als Web-App – aber mit einer vollständigen, professionellen **CI/CD-Pipeline auf AWS**. Der Fokus liegt nicht auf der App, sondern auf der DevOps-Automatisierung: Jede Code-Änderung wird automatisch getestet und ausgerollt.

So läuft es (Pipeline in einem Satz)

git push auf main → GitHub Actions führt die Tests aus → nur wenn sie grün sind (Test-Gate) → Deploy auf die **Staging-EC2**. Ein **Tag v1.x.x** löst das **Production-Deployment** aus. Slack meldet Start, Erfolg und Fehler.

```
Code (GitHub) --> GitHub Actions: Test --> Docker Build --> EC2 (AWS)
Terraform baut die Infrastruktur: VPC -> Subnetz -> Security Group -> EC2
Remote State: S3-Bucket | Benachrichtigung: Slack Webhook
```

Tech-Stack

Bereich	Technologie	Wo im Repo
Frontend	React (Vite)	frontend/
Backend	FastAPI (Python) + SQLite	backend/
Tests	pytest (10 Tests, In-Memory-DB)	backend/test_main.py
Container	Docker Multi-Stage, nginx:alpine	*/Dockerfile
Infrastruktur	Terraform (VPC, EC2, S3)	infra/
CI/CD	GitHub Actions	.github/workflows/

Alle wichtigen Links (klickbar)

GitHub-Repository (der gesamte Code)

<https://github.com/danialahmed2207/cloudops-capstone>

GitHub Actions (Pipeline-Läufe ansehen)

<https://github.com/danialahmed2207/cloudops-capstone/actions>

Staging-Workflow (die Pipeline-Definition)

<https://github.com/danialahmed2207/cloudops-capstone/blob/main/.github/workflows/deploy-staging.yml>

AWS Console – Region Frankfurt (eu-central-1)

<https://eu-central-1.console.aws.amazon.com/console/home?region=eu-central-1>

Lokal: Frontend im Dev-Modus (wenn npm run dev läuft)

<http://localhost:5173>

Lokal: API-Dokumentation Swagger (wenn uvicorn läuft)

<http://localhost:8000/docs>

Lokal: kompletter Docker-Stack (wenn docker compose läuft)

<http://localhost:8080>

Wie starte ich was? (Spickzettel)

Backend lokal (Terminal 1):

```
cd ".../cloudops-capstone/backend"
source ~/.venvs/cloudops-backend/bin/activate
uvicorn main:app --reload # → localhost:8000/docs
pytest # Tests ausführen
```

Frontend lokal (Terminal 2):

```
cd ".../cloudops-capstone/frontend"
npm run dev # → localhost:5173
```

Docker-Gesamtstack (wie in Produktion):

```
cd ".../cloudops-capstone"
./scripts/docker-up.sh # → localhost:8080
```

Infrastruktur (Terraform):

```
cd ".../cloudops-capstone/infra"
terraform plan # zeigt, was angelegt würde
terraform apply # legt es wirklich an
terraform destroy # räumt alles ab (Kosten sparen!)
```

Projekt-Status

- [x] 1. Backend (FastAPI) lokal lauffähig, pytest grün (10/10)
- [x] 2. Frontend (React/Vite) mit Anbindung ans Backend
- [x] 3. Docker Multi-Stage für Frontend und Backend
- [~] 4. Terraform-Infra (Dateien fertig und validiert; es fehlen IAM-Rechte für plan/apply)
- [] 5. GitHub Actions Staging komplett (Secrets + echtes Deploy)
- [] 6. GitHub Actions Production (Tag v*) + Slack
- [] 7. Doku, Architektur-Diagramm, Kostenkapitel (IHK)

Wichtige Begriffe (fürs Verständnis / die Präsentation)

Test-Gate: Der Deploy-Job hat 'needs: test' – er startet nur, wenn die Tests grün sind. Kaputter Code erreicht nie den Server.

Multi-Stage-Build: Das Docker-Image wird in zwei Stufen gebaut: Stufe 1 kompiliert (Node), Stufe 2 enthält nur das Ergebnis + Nginx. Ergebnis: kleines, sicheres Produktions-Image ohne Quellcode.

tfstate (Terraform State): Terraforms Gedächtnis: welche Ressourcen es angelegt hat. Liegt im S3-Bucket, damit Laptop und CI denselben Stand sehen – und nie im Git-Repo.

Staging vs. Production: Staging = Testumgebung, wird bei jedem Push auf main aktualisiert. Production = Live-Umgebung, nur bei einem Versions-Tag (v1.0.0).

Secrets: Zugangsdaten (AWS-Keys, SSH-Key, Slack-URL) liegen verschlüsselt in GitHub – niemals im Code.